Runtime

Version: 1.0

Purpose

This document describes the runtime architecture of Trading Platform Pro.
The runtime layer is responsible for starting, operating, monitoring and shutting down the application in a controlled and deterministic manner.

Objectives

The runtime shall provide:
• deterministic startup
• deterministic shutdown
• centralized initialization
• service orchestration
• lifecycle management
• runtime monitoring
• fault isolation

Runtime Lifecycle

The application lifecycle consists of the following phases:
```
Bootstrap
↓
Configuration
↓
Dependency Injection
↓
Infrastructure Initialization
↓
Application Initialization
↓
Runtime Start
↓
Operational State
↓
Shutdown
```
Each phase has a clearly defined responsibility.

Startup Sequence

Application startup should always follow the same order.
1. Load configuration
2. Initialize logging
3. Build dependency container
4. Register infrastructure services
5. Register application services
6. Validate configuration
7. Start runtime services
8. Enter operational mode
No application logic should execute before initialization is complete.

Runtime Services

Typical runtime services include:
• Scheduler
• Event Dispatcher
• Health Monitoring
• Metrics
• Clock Services
• Plugin Manager
• Background Workers
All runtime services should be managed centrally.

Lifecycle Management

Every runtime component should support:
• initialize()
• start()
• stop()
• dispose()
Shutdown should always occur in reverse startup order.

Error Handling

Runtime failures should:
• be logged
• remain isolated where possible
• avoid cascading failures
• support graceful shutdown
Unexpected exceptions must never terminate the application without logging.

Monitoring

The runtime continuously monitors:
• service status
• background workers
• scheduler
• event processing
• resource usage
• critical failures
Monitoring should support diagnostics without affecting business logic.

Graceful Shutdown

Shutdown should:
1. stop accepting new work
2. finish running tasks where appropriate
3. flush logs
4. release resources
5. terminate cleanly

Performance

Runtime services should:
• avoid unnecessary allocations

• minimize blocking operations
• remain responsive
• support future scalability

Runtime Resilience

The runtime shall:
• recover from transient failures where possible
• isolate failing services
• support graceful degradation
• preserve deterministic behaviour
Critical failures shall trigger controlled shutdown procedures.

Runtime Review Checklist

Before release verify:
• startup sequence validated
• shutdown sequence validated
• lifecycle managed correctly
• monitoring active
• logging integrated
• documentation updated

Related Documents

• Architecture.md
• Infrastructure.md
• Configuration.md
• Logging.md
• Monitoring.md
• AGENTS.md